

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (Bsc.IT Semester III)**  
**Data Structures Practical**

## Practical - VII

|                                 |                                      |
|---------------------------------|--------------------------------------|
| Roll No. : S033                 | Name: Rudra Rawal                    |
| Class : SYIT                    | Batch : 1                            |
| Date of Assignment : 2026-08-05 | Date/Time of Submission : 2026-08-05 |

Hash Table:

Write a program to:

- Implement a hash table with separate chaining for collision handling.

CODE:

```
*Practical_7A.py - C:/Users/mvlui/Desktop/SYIT_033/DS/Practical_7A.py (3.14.6)*
File Edit Format Run Options Window Help
size = 5
table = [[] for _ in range(size)]

def insert(key, value):
    table[(key % size)].append((key, value))

insert(10, "Apple")
insert(15, "Banana")
insert(20, "Cheery")
insert(32, "Guava")
insert(33, "Mango")

print(table)

print("\nRudra_S033")
```

OUTPUT:

```
*Practical_7A.py - C:/Users/mvlui/Desktop/SYIT_033/DS/Practical_7A.py (3.14.6)*
[[ (10, 'Apple'), (15, 'Banana'), (20, 'Cheery') ], [], [ (32, 'Guava') ], [ (33, 'Mango') ], [] ]
Rudra_S033
```

b. Store and retrieve data from the hash table.

CODE:

```
Practical_7A.py - C:/Users/mvlui/Desktop/SYIT_033/DS/Practical_7A.py (3.14.6)
File Edit Format Run Options Window Help

size = 5
table = [[] for _ in range(size)]

def insert(key, value):
    table[(key % size)].append((key, value))

def search(key):
    for k,v in table[key % size]:
        if k == key:
            return v
    return "Not Found"

insert(10, "Apple")
insert(15, "Banana")
insert(20, "Cheery")
insert(32, "Guava")
insert(33, "Mango")

print(table)

key = int(input("Enter Key: "))
print(search(key))

print("\nRudra_S033")
```

OUTPUT:

```
===== RESTART: C:/Users/mvlui/Desktop/SYIT_033/DS/Practical_7A.py =====
[[ (10, 'Apple'), (15, 'Banana'), (20, 'Cheery') ], [], [(32, 'Guava')], [(33, 'Mango')], []]
Enter Key: 30
Not Found

Rudra_S033

===== RESTART: C:/Users/mvlui/Desktop/SYIT_033/DS/Practical_7A.py =====
[[ (10, 'Apple'), (15, 'Banana'), (20, 'Cheery') ], [], [(32, 'Guava')], [(33, 'Mango')], []]
Enter Key: 10
Apple

Rudra_S033

===== RESTART: C:/Users/mvlui/Desktop/SYIT_033/DS/Practical_7A.py =====
[[ (10, 'Apple'), (15, 'Banana'), (20, 'Cheery') ], [], [(32, 'Guava')], [(33, 'Mango')], []]
Enter Key: 32
Guava

Rudra_S033
```

### Curiosity Questions:

1. In a BST, which side (left or right) has **smaller numbers** than the root?
2. If we insert numbers 10, 20, 5 into a BST, which number will be at the root?
3. What will the inorder traversal of a BST give — sorted numbers or random numbers?
4. Where will the **largest number** in a BST always be found?
5. If the root of a BST is 50 and we search for 70, will we go left or right? Why?
6. If a BST has only **one node**, what is its inorder traversal?
7. Can we create a BST from characters like A, B, C instead of numbers?
8. If we insert numbers 5, 10, 15, 20 in order, will the tree be balanced or like a straight line?
9. In a BST, do we check the left child first or the right child first in inorder traversal?
10. If we delete the root of a BST, will the tree disappear?